

A campaign-level dataset of 401 GHSA and OSV malware advisories for 200 PyPI and npm packages, with mechanism labels and reporter-coverage annotations

Tahsin Tajwar Tanni^{a,*}, Nafiz Khan^a

^a*Independent Researcher, Dhaka, Bangladesh*

Abstract

This dataset is a frozen, checksummed snapshot of 401 confirmed-malicious package advisories for the PyPI and npm ecosystems, collected on 2 October 2026 from the GitHub Security Advisory database (GHSA, 200 records) and OSV.dev (201 records). The 401 records describe 200 distinct packages (100 per ecosystem); 386 of them were published between 1 September and 2 October 2026. Each record carries the advisory identifier, source, package name, ecosystem, summary, full description text, publication timestamp, severity, reference URLs and fetch timestamp. The dataset is accompanied by derived tables produced by a documented, tested pipeline: (i) a two-stage collapse of records into 94 independent attack campaigns (per-package merge of cross-source re-reports, then grouping by reporter-assigned campaign tag or masked description prefix); (ii) a binary *described-mechanism* label per record from a six-category regular-expression taxonomy with a negation guard, together with a 51-row human verification sheet (precision 0.94, 95% CI 0.83–0.98); (iii) per-package reporter-coverage annotations parsed from the advisories' **## Source:** headers (reporter count and indicators for kam193, Amazon Inspector and GHSA boilerplate); and (iv) a compound/generic naming flag per package, with its validation against 139 publicly disclosed LLM-hallucinated package names and the top-5,000 packages of each ecosystem. The data support studies of advisory duplication, reporter bias in advisory text, malicious-package naming and the reuse of advisory databases as ground truth. All code, tests and manifests needed to regenerate every

*Corresponding author.

Email address: CORRESPONDING-EMAIL@example.com (Tahsin Tajwar Tanni)

derived table from the frozen snapshot are included.

Keywords: software supply chain, malicious packages, security advisories, PyPI, npm, GHSA, OSV, measurement validity

Specifications Table

Subject	Computer Science: Software.
Specific subject area	Software supply-chain security; malicious open-source packages; security advisory databases; measurement of advisory text as a behavioural label.
Type of data	Table (JSON Lines, JSON, CSV, Markdown); text (advisory descriptions); processed and analysed derived tables; Python source code and tests. Raw data: advisory records as returned by the GHSA and OSV APIs, deduplicated and frozen. Derived data: campaign assignments, mechanism labels, reporter-coverage annotations, naming-grammar flags, human verification sheet, grammar validation statistics, power-analysis outputs.
Data collection	Advisories of type malware for the pip and npm ecosystems were retrieved from the GitHub Security Advisory (GHSA) REST API (five pages, newest first, unauthenticated) on 2 October 2026. For every package name found, OSV.dev was queried for its corresponding advisory. Records were deduplicated on (ecosystem, package name, advisory identifier) and written to a single JSON Lines file with a SHA-256 manifest. Labels and annotations were computed from that file only, by scripts included in the repository, using Python 3.14 with pandas, scipy and statsmodels. One author verified 51 labeler positives by reading the advisory text under a written rule.

Data source location	Primary sources: GitHub Security Advisory database (https://github.com/advisories , GitHub, Inc., San Francisco, USA) and OSV.dev (https://osv.dev , Google LLC, Mountain View, USA). Secondary source for grammar validation: the publicly disclosed “universal” hallucinated-package-name lists of Churilov [1], release v0.2-preprint , Zenodo DOI 10.5281/zenodo.19859120 (CC BY 4.0). Benign comparison lists: top-5,000 PyPI downloads (hugovk/top-pypi-packages) and npm-high-impact v1.13.0. Data were collected and processed in Dhaka, Bangladesh.
Data accessibility	Repository name: Zenodo. Data identification number: 10.5281/zenodo.XXXXXXX [TODO: mint before submission]. Direct URL to data: https://doi.org/10.5281/zenodo.XXXXXXX Source code (mirror): https://github.com/TahsinTanni/Phantom-Guard Instructions for accessing these data: download the Zenodo archive; <code>data/frozen/incidents_snapshot.jsonl</code> is the raw snapshot; <code>python run_check.py</code> regenerates the campaign-level tables from it; <code>python -m pytest tests/ -q</code> runs the test suite.
Related research article	None. [TODO: if the companion analysis paper is accepted before this article is submitted, cite it here as the related research article.]

1. Value of the Data

- Advisory databases are increasingly mined as ground truth for what malicious packages do, yet their records are not independent observations: this dataset makes the duplication explicit (401 records → 200

packages $\rightarrow$ 94 campaigns) and provides the campaign assignments, so researchers can run tests at the correct unit of analysis.

- Every advisory text is annotated with the reporters that contributed to it. This allows the effect of reporter coverage on text-derived labels to be measured and controlled, which no public malicious-package dataset currently supports.
- The snapshot is frozen and checksummed, and every derived table is regenerated from it by tested code, so results computed on it are exactly reproducible even though both source databases change continuously.
- The 51-row human verification sheet and the written labeling rule give a measured precision for the regex-based mechanism labels, which lets other researchers decide whether to reuse the labels or re-annotate.
- The naming-grammar validation (hallucinated versus benign versus malicious name sets) is a reusable negative result: a compound-token grammar does not identify LLM-hallucinated package names, which saves others from building on that proxy.

2. Background

Malicious packages make up a large share of reported vulnerabilities in npm and PyPI [2], and large language models add a further route to them by recommending packages that do not exist, whose names an attacker can then register [3, 4]. Empirical work on this threat commonly treats the GitHub Security Advisory database (GHSA) and OSV as ground truth both for which packages are malicious and for what the malware does. Two assumptions are usually made and rarely tested: that advisory records are independent, and that a mechanism mentioned in the advisory text is a property of the malware rather than of whoever wrote the advisory.

This dataset was assembled to test those assumptions while studying whether compound/generic package naming (e.g. `py-*`, `*-tools`, `*-client`) is associated with a described payload mechanism. During that study, the campaign grouping was corrected three times (142 $\rightarrow$ 106 $\rightarrow$ 94 campaigns) as the human verification surfaced merge failures, and the dependence of labels on reporter coverage emerged as the dominant effect. We release the snapshot, the corrected grouping, the labels, the reporter annotations and

the verification sheet so that these measurement properties can be reused and the analysis independently repeated or extended.

3. Data Description

The repository is organised as a Python project. The files that constitute the dataset are listed below; all paths are relative to the repository root.

3.1 Raw snapshot

`data/frozen/incidents_snapshot.jsonl` — 401 records, one JSON object per line. Table 2 gives the fields. Table 3 summarises the composition and Fig. 1 shows the publication dates.

Table 2: Fields of `incidents_snapshot.jsonl`.

Field	Type	Content
<code>advisory_id</code>	string	Advisory identifier: <code>GHSA-xxxx-xxxx-xxxx</code> (GHSA), <code>MAL-yyyy-nnnn</code> (OSV malicious-package record), or <code>PYSEC-...</code> (one OSV record).
<code>source</code>	string	<code>ghsa</code> or <code>osv</code> .
<code>package_name</code>	string	Package name as registered, including npm scope where present.
<code>ecosystem</code>	string	<code>pypi</code> or <code>npm</code> .
<code>summary</code>	string	One-line advisory summary.
<code>description</code>	string	Full advisory text (Markdown). Median length 1,335 characters (range 216–4,414). OSV texts concatenate per-reporter sections introduced by <code>## Source:</code> headers.
<code>published_at</code>	string	ISO-8601 publication timestamp (UTC).
<code>withdrawn_at</code>	string	Withdrawal timestamp; <code>None</code> for all 401 records.
<code>severity</code>	string	<code>critical</code> for GHSA records; empty for OSV records, which carry no severity.
<code>reference_urls</code>	list of strings	Reference URLs attached to the advisory (reporter pages, registry pages).
<code>fetches_at</code>	string	ISO-8601 retrieval timestamp (UTC).

Table 3: Composition of the raw snapshot.

Records		401
from GHSA / from OSV		200 / 201
PyPI / npm		201 / 200
Distinct packages (ecosystem, name)	200 (100 PyPI, 100 npm)	
Published September 2026 / October 2026		304 / 82
Published before September 2026	15 (earliest 30 October 2025)	
Withdrawn		0

Every package has exactly one GHSA record and one OSV record, except one PyPI package that has two OSV records (a MAL- and a PYSEC- entry); that is why there are 401 rather than 400 records. The accompanying file `data/metadata/incidents_snapshot_manifest.json` records the SHA-256 digest of the snapshot, the record count, the retrieval date, the preprocessing version and the installed package versions.

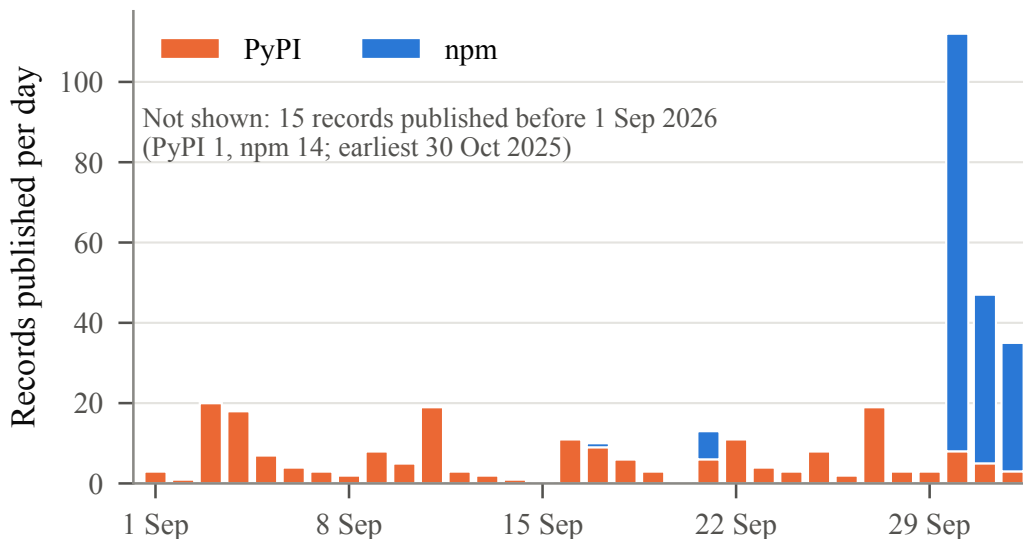


Figure 1: Advisory records published per day, 1 September to 2 October 2026, by ecosystem. The npm records cluster in the last three days of the window.

3.2 Derived campaign-level data

The two-stage collapse described in Section 4.3 assigns every package to one of 94 campaigns (22 npm, 72 PyPI; Fig. 2). The campaign table is

regenerated by `python run_check.py` and written to `results/`; it carries, per campaign, the member packages, the ecosystem, the campaign signature, the aggregated naming-grammar flag, the aggregated described-mechanism label, the reporter count and the three reporter indicators. The npm side is dominated by a few families: two WhatsApp-library fork families of 45 and 15 names, a 10-name dependency-confusion family and two related families of 10 and 3 names, which together contain 83 of the 100 npm packages. PyPI campaigns are mostly singletons.

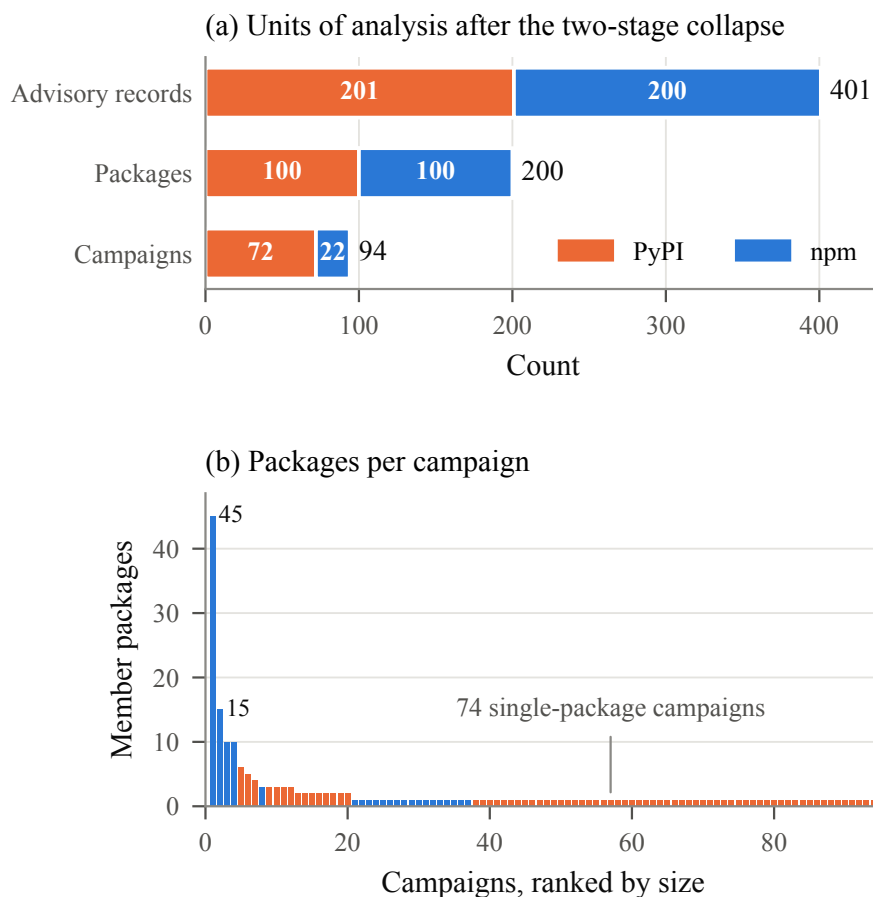


Figure 2: (a) Records, packages and campaigns by ecosystem. (b) Member packages per campaign, ranked by size; 74 of the 94 campaigns contain a single package.

3.3 Mechanism labels and human verification

`src/labeling/malware_taxonomy.py` defines six mechanism categories (obfuscated payload execution; hidden install hook; credential or wallet exfiltration; remote backdoor access; remote payload retrieval; brand-impersonation narrative), each as a list of regular expressions with the advisory text that motivated it. The labeler in `src/labeling/malware_labeler.py` reads text only, never the package name, which a test enforces. Of the 94 campaigns, 51 are labelled positive under the union of a package’s GHSA and OSV texts; Fig. 3 gives the count per category.

`reports/labeler_spot_check.md` is the human verification sheet: 51 labeler-positive rows (all 21 packages that are positive only through OSV-carried text, plus 30 sampled campaigns), each with the matched span, the surrounding text, the annotator’s verdict (REAL / FALSE / UNSURE) and a note. `results/spot_check_verdicts.json` holds the same verdicts in machine-readable form. The outcome was 44 REAL, 3 FALSE and 4 UNSURE: precision 0.936 (95% Wilson CI 0.828–0.978) excluding UNSURE, or 0.863 (0.743–0.932) counting UNSURE as FALSE. `reports/osv_only_positives.md` lists the 21 OSV-only positives with their reporter sections.

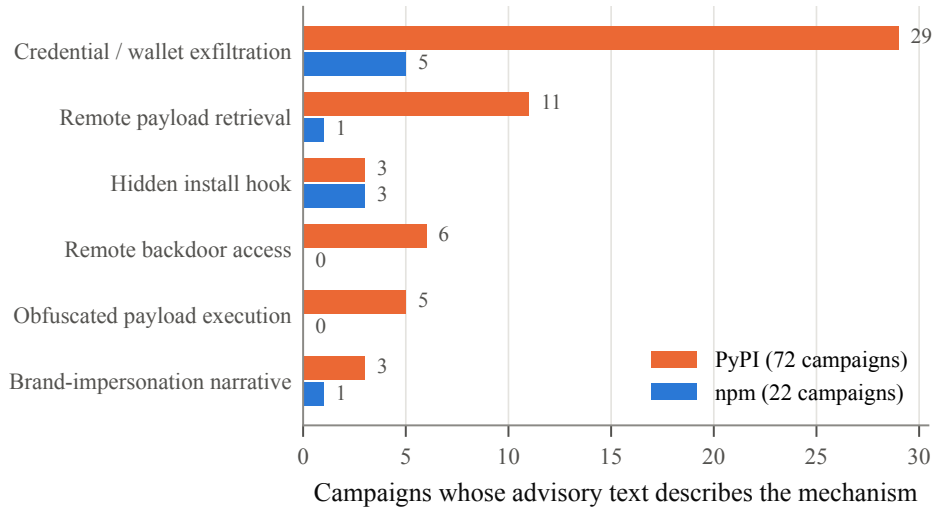


Figure 3: Campaigns whose advisory text (GHSA and OSV combined) describes each mechanism category, by ecosystem. A campaign can match several categories.

3.4 Reporter-coverage annotations

For every package, the `## Source:` headers of both texts are parsed (`extract_reporters` in `src/data/incident_clients.py`) into a reporter count (`n_reporters`) and three indicators: `kam193`, `amazon_inspector` and `boilerplate` (GHSA’s generic “should be considered fully compromised” text, which OSV mirrors as a `ghsa-malware` section; the two count as one reporter). Headerless text without boilerplate is marked `<unattributed>` (73 of 200 packages). Fig. 4 shows the reporter combinations per package. Among campaigns, the described-mechanism rate (Fig. 5) is 0.65 (46/71) with Amazon Inspector coverage against 0.22 (5/23) without, and 0.38 (5/13) for campaigns in which the GHSA boilerplate is one of the reporters; the 4 campaigns whose only reporter is the boilerplate are all mechanism-negative (0/4).

3.5 Naming-grammar flag and its validation

`src/data/naming_grammar.py` flags a name that contains a token from lists of compound prefixes, compound suffixes or trend suffixes (e.g. `py-`, `-tools`, `-client`, `-sdk`). `results/grammar_validation.json` reports the flag rate on four sets (Table 4, Fig. 6). `data/external/hallucinated_names.txt` is the 139-name validation set (121 PyPI, 18 npm) with its provenance in `data/external/hallucinated_names_SOURCES.md`.

Table 4: Naming-grammar flag rate by set, with Wilson 95% confidence intervals.

Set	<i>n</i>	Flagged	Rate [95% CI]
LLM-hallucinated names [1]	139	7	0.050 [0.025, 0.100]
Benign top-5,000 PyPI	5,000	821	0.164 [0.154, 0.175]
Benign top-5,000 npm	5,000	1,031	0.206 [0.195, 0.218]
Malware campaigns (this dataset)	94	25	0.266 [0.187, 0.363]

3.6 Analysis outputs

`results/power_campaign_level.json` contains the pooled and per-ecosystem 2×2 tables, Fisher and Cochran–Mantel–Haenszel results and the power analysis. For reference, the pooled table of grammar flag against described mechanism is $[[13, 12], [38, 31]]$, odds ratio 0.884 (95% CI 0.353–2.211), and the design has power 0.31 to detect an odds ratio of 2.0 at $n = 94$.

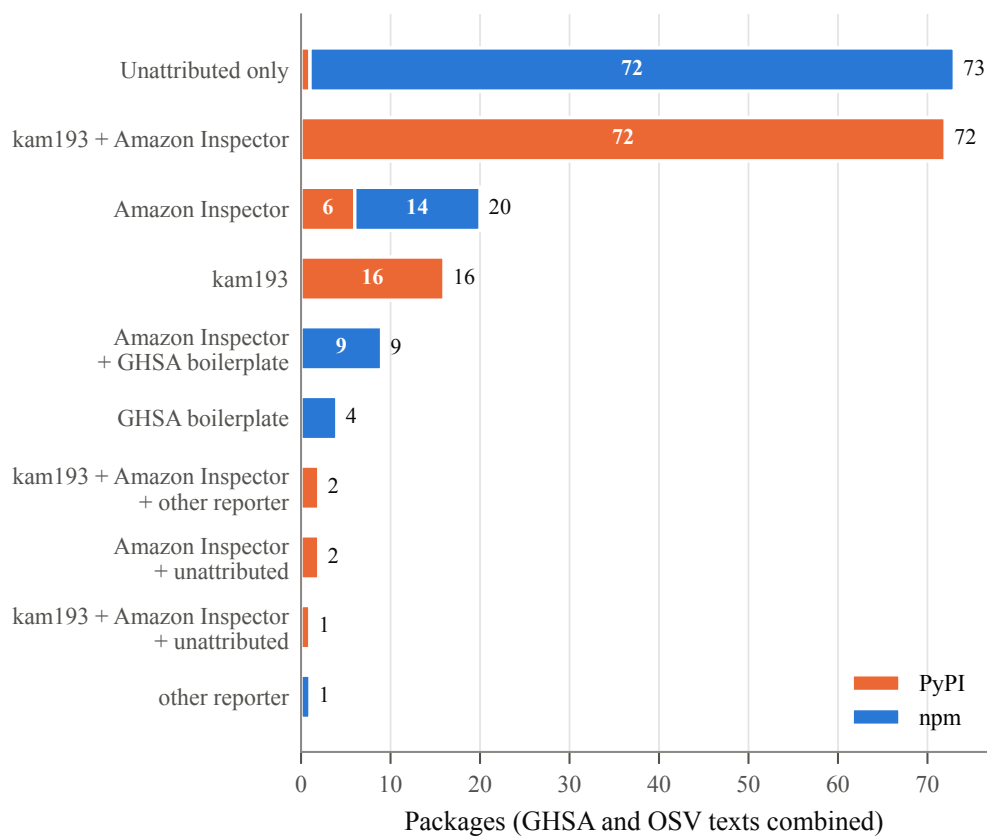


Figure 4: Reporter combinations per package, parsed from the `## Source:` headers of both texts. Most npm packages carry no reporter header; most PyPI packages are covered by both kam193 and Amazon Inspector.

`reports/research_log.md` is the numbered, dated log of every pipeline change and the numbers it produced, including the superseded groupings.

4. Experimental Design, Materials and Methods

4.1 Collection

The GHSA REST API was queried for advisories with `type=malware` and ecosystem `pip` or `npm`, unauthenticated, newest first, five pages per ecosystem, on 2 October 2026. GHSA is a live API; the five-page window is the reason 386 of 401 records were published in the five weeks before retrieval. For each package name in the GHSA result, OSV.dev was

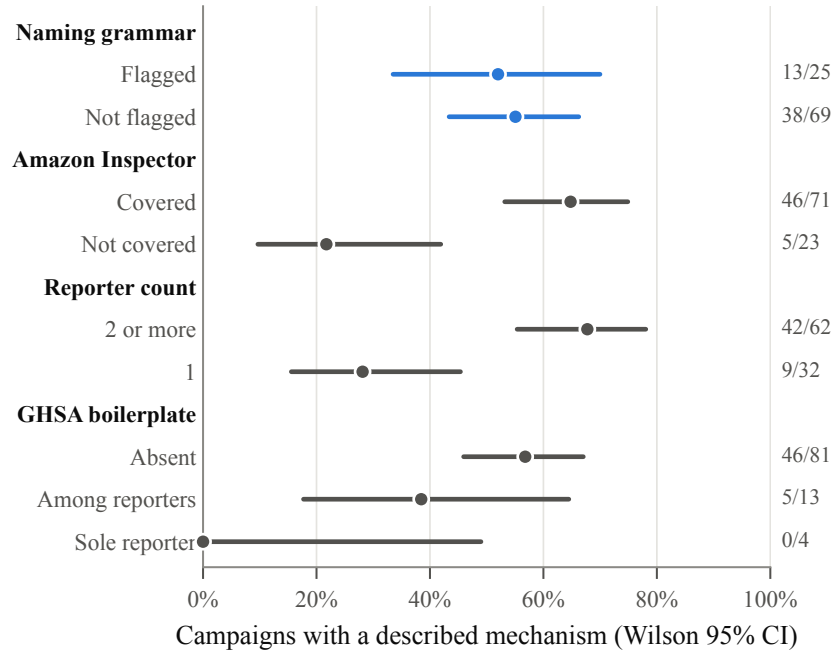


Figure 5: Share of campaigns labelled mechanism-positive, by naming flag and by reporter coverage (Wilson 95% CIs; counts at right).

queried by (ecosystem, package) and the returned advisory record stored. Retrieval is checkpointed and resumable (`src/data/collect_incidents.py`, `src/data/incident_clients.py`).

4.2 Deduplication and freezing

Records were deduplicated on the key (ecosystem, package name, advisory identifier), preferring GHSA on an exact-key collision and keeping distinct advisory identifiers from either source as separate records. The result was written once to `incidents_snapshot.jsonl` and a manifest with its SHA-256 digest was generated (`src/utis/manifest.py`). All later steps read only this file; the repository’s working rules forbid modifying it, and new data must go in a new file with its own manifest.

4.3 Campaign collapse

Statistical tests on advisories assume independent units. Without collapsing, one npm table in the development history reached an odds ratio of 178 on single-digit cells, driven by one attacker’s

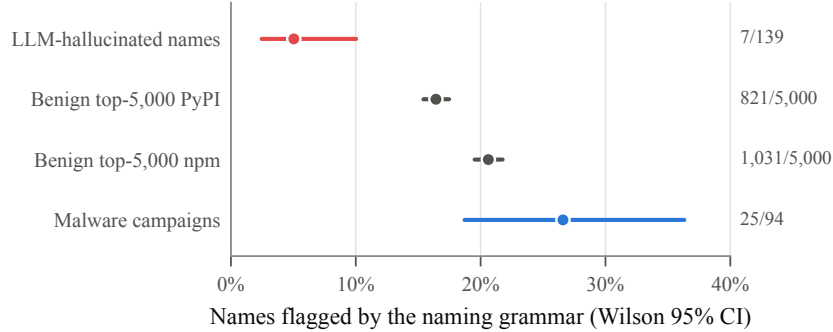


Figure 6: Naming-grammar flag rate by set, with Wilson 95% confidence intervals.

many package names. The collapse (`collapse_to_campaign_level` in `src/statistics/h1_pilot_analysis.py`) has two stages. Stage 1 merges all records of the same (ecosystem, package), which removes GHSA/OSV re-reports. Stage 2 groups packages by a campaign signature chosen in priority order: (i) the reporter’s own **Campaign:** identifier when present (assigned by the kam193 reporter); (ii) otherwise the first 200 characters of the description after stripping **## Source:** headers and masking the package’s own name at word boundaries. GHSA’s generic boilerplate and kam193’s generic install-time pentest template are never used as keys because they are identical across unrelated incidents; packages carrying only such text remain their own unit. A campaign counts as grammar-flagged, or as mechanism-positive, if any member does. The signature rules were revised twice after human reading of sampled rows showed that per-package headers made identical texts look different and that a 10-name family whose texts begin with the package name was never merged; the superseded groupings (142 and 106 campaigns) are preserved in the research log.

4.4 Mechanism labeling

The six-category taxonomy was derived from advisory text, first from PyPI examples and then extended with npm phrasing (post-install hooks, JavaScript obfuscators, verbs such as *reads* and *POSTs*) after the PyPI-derived patterns matched 0 of about 200 npm records. A negation guard drops a match when a cue (*does not*, *do not*, *did not*, *doesn’t*, *no evidence of*) occurs within 40 characters before it, unless *but*, a semicolon or a period intervenes. The label used for the campaign table is computed on the union of a package’s GHSA and OSV descriptions; a GHSA-only variant is available through

`run_check.py -text-source ghsa`. Generic scanner-verdict boilerplate is deliberately not matched, and a regression test enforces this.

4.5 Human verification

One author read 51 labeler positives under a written rule recorded in the sheet: a match counts as REAL if the surrounding prose describes the mechanism, as FALSE if the match occurs only in an analyst tag list and not in prose, and as UNSURE otherwise. The sample comprised all 21 OSV-only positives, a census of 12 npm campaigns and a seeded random draw of 18 PyPI campaigns. The annotator saw package names; recall was not measured.

4.6 Reporter extraction and grammar validation

Reporter sections were parsed from the **## Source:** headers of both texts of each package and merged per campaign. The grammar was validated by running it unchanged over the 139 universal hallucinated names of Churilov [1], the top-5,000 PyPI packages by downloads and the top-5,000 npm packages by dependents; Wilson intervals are reported. An earlier npm control drawn from registry search results for common terms was flagged at 73% and discarded, because the search terms were themselves grammar tokens.

4.7 Software

Python 3.14; pandas, scipy, statsmodels, requests. The test suite (`tests/`, 160 tests) covers the clients, deduplication, manifest generation, grammar, labeler (including the name-blindness and boilerplate rules), collapse and reporter extraction. `run_check.py` reproduces the campaign-level tables and the power analysis from the frozen snapshot in one command.

Limitations

The snapshot covers a five-week publication window and 200 packages, because GHSA returns advisories newest first and five pages were fetched; it is a temporal convenience sample, not a census. The npm side consists of a few large families plus singletons, so npm-level statistics rest on very few independent units. Mechanism labels come from a regular-expression taxonomy verified by a single annotator on 51 positives, with no inter-annotator agreement and no recall estimate. Reporter coverage is annotated from headers and 37% of packages remain unattributed. The campaign grouping was corrected three times and further under-merging is possible. The naming-grammar flag

measures compound/generic naming, not LLM hallucination. All packages are confirmed malicious; the benign comparison sets are name lists only and are not matched to the malware set.

Ethics Statement

The authors have read and follow the ethical requirements for publication in *Data in Brief*. The data consist of publicly published security advisories and package names; no human subjects, animals or personal data are involved. The dataset contains no malicious code, only advisory text describing it. GHSA advisory data are licensed CC BY 4.0; OSV records carry per-source licences noted in each record’s reference URLs; the hallucinated-name list is CC BY 4.0.

CRedit authorship contribution statement

Tahsin Tajwar Tanni: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Nafiz Khan:** Conceptualization, Methodology, Software, Investigation, Writing – review & editing.

Acknowledgements

This work received no external funding.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

Phantom Guard: frozen GHSA/OSV malware-advisory snapshot and campaign-level derived data (Original data) (Zenodo), <https://doi.org/10.5281/zenodo.XXXXXXX>.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to assist with code development and with drafting text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] A. Churilov, The range shrinks, the threat remains: Re-evaluating LLM package hallucinations on the 2026 frontier-model cohort, arXiv:2605.17062 (2026). Data: <https://doi.org/10.5281/zenodo.19859120>
- [2] S. A. Akhavani, B. Ousat, A. Kharraz, Open source, open threats? Investigating security challenges in open-source software, arXiv:2506.12995 (2025). <https://doi.org/10.48550/arXiv.2506.12995>
- [3] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, M. Jadliwala, We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs, in: Proc. 34th USENIX Security Symposium, 2025. arXiv:2406.10279.
- [4] Z. Huang, C. Liu, Z. Wang, X. Cui, X. Deng, M. Xu, Package hallucinations as phantoms in open-source software supply chains: An empirical security analysis, ACM Trans. Softw. Eng. Methodol. (2026).